From mid-project to final presentation

Editing guide · 17 October 2026
Based on the actual 16-slide deck: `MCP Server Security Scanner - Mid-Project Presentation.pptx`. The original PowerPoint is preserved. This document specifies the revisions; it is not a replacement slide deck.

The final presentation's central message

> We built an MCP static security scanner with 16 threat categories, actionable reports, and conservative remediation. Targeted improvements increased recall on development cases, while reserved validation revealed a specific tool-extraction gap that defines the next step.

Present the project as a delivered engineering system with an evidence-backed evaluation. Lead with what a reviewer can run and inspect. Keep the distinction between implemented scope and measured effectiveness visible. Use Final_Project_Results.html as the results handout.

Assume a **12-minute presentation plus questions** until the course duration is confirmed. The proposed main deck has 15 slides; detailed software diagrams and per-rule tables go in an appendix. Preserve the existing university branding and visual theme, using consistent type sizes and no more than one main conclusion per slide.

Changes to every existing slide

Current slide	Action	Required change
1 · MCP Server Security Scanner	Keep and update	Add “Final Project” and the presentation date. Preserve students, supervisor, and lab attribution. Subtitle: “Static analysis, actionable reports, and reproducible evaluation.”
2 · Model Context Protocol (MCP)	Shorten; merge with 3	Keep host/client/server roles, but reduce definitions. Replace the linear “LLM → Host → Client → MCP Server” text with host containing LLM + client, client ↔ server. Mark server source as the scanner's input.
3 · Inside an MCP Server	Merge into 2	Keep tools/resources/prompts as a compact context panel. Avoid implying equally deep extraction for all three; tool metadata and supported source patterns are the principal detection surfaces.
4 · Security Challenges of MCP Servers	Keep; make concrete	Show a tool description or external response influencing an assistant. Tie impact to secret exposure and unsafe execution. Remove broad comparisons with other scanners unless supported by a direct comparison study.
5 · Targeted Security Threats	Replace scope claim	Delete “Current implementation focuses on detecting these two primary threats.” State “16 catalog categories / 25 rules; deepest detection and eligible automatic fixes in MCP03 and MCP06.” Catalog breadth is implementation scope, not proof that all categories were validated.
6 · Why an MCP Security Scanner?	Merge into delivered workflow	Show source path → static scan → evidence → review/remediation. Keep “without executing the server.” Replace unmeasured claims of reduced review effort with the concrete workflow available to the user.
7 · Project Objectives	Convert to achievements	Use “Objective → delivered evidence”: parsing → supported source patterns; detection → catalog; mitigation → two eligible fix categories; reporting → console/JSON/HTML; evaluation → frozen runs and recorded metrics. Do not mark universal detection as complete.
8 · High-Level Architecture	Keep and fill placeholder	Draw target source/config/dependencies → discovery → parser/model → analyzers → filter/deduplicate → reports. Add an optional remediation branch. The existing slide contains a diagram placeholder, not a completed architecture figure.
9 · Internal Architecture	Merge into 8 or appendix	Replace placeholder with real module responsibilities. Avoid spending another main-deck slide repeating the same pipeline.

Current slide	Action	Required change
10 · Data Flow	Repurpose as worked detection example	Replace placeholder with external text → handler return → MCP06 finding, alongside one concrete source excerpt and evidence location. Describe static evidence of risk; do not claim an observed LLM compromise.
11 · Component Diagram	Move to appendix	Fill only if useful for technical questions. Connect scanner, parsers, analyzers, reporters, and fixer using actual modules. Reclaim main-deck time for evaluation.
12 · Detection Algorithms	Update substantially	Keep Python AST and metadata signatures; add bounded flow/alias analysis and supported JS/TS registration patterns. Explain one precision improvement and one recall improvement. JS/TS support is structural and limited, not a full compiler analysis.
13 · Software Components	Merge with architecture appendix	The six component descriptions duplicate slides 8–11. Replace the class-diagram placeholder if retained; show real classes/modules instead of a speculative UML design.
14 · Technologies Used	Update; move to appendix	Its table currently lists Python, AST, regex, and JSON/console. Add standard-library core, JSON/config parsing, HTML reporting, and reproducible evaluation artifacts. Optional AI review must be labeled optional and unevaluated in the reported deterministic runs.
15 · Current Progress & Next Steps	Replace with results + limitations	Remove the old to-do list for additional detectors, mitigation, reporting, and final testing. These have delivered implementations. Add measured results and a separate remaining-work slide prioritizing handwritten JSON-RPC extraction.
16 · Thank You	Replace ending, retain Q&A	End with three delivered contributions and one precise next step; leave Q&A as a small footer. The final spoken message should be the project's contribution.

Proposed final deck: ready-to-use slide content

1 · MCP Server Security Scanner — Final Project

On slide: “Static analysis, actionable reports, and reproducible evaluation.” Retain Noam Ben-Lulu, Bar David, Sorana Fraier, NSSL, and Technion attribution.

Say: “Our project takes an MCP server's source code and turns it into reviewable security findings without running that server.” **Time:** 15 seconds.

2 · Where MCP introduces a security boundary

On slide: Host containing assistant/LLM and MCP client ↔ MCP server → external data and services. Add the tools/resources/prompts context from old slide 3. Mark the scanner inspecting server source offline.

Say: “Tool metadata and returned content can influence an assistant. Server code also controls access to sensitive operations.” **Time:** 35 seconds.

3 · A small description can change a large action

On slide: A benign tool description beside a short illustrative instruction to disclose a secret. Highlight the instruction, then the risk. Use fictitious data; label the snippet as illustrative.

Say: “We inspect suspicious instructions and unsafe data paths as static evidence. We are not proving that a live assistant was successfully exploited.” **Time:** 35 seconds.

4 · What we delivered

On slide: “16 categories · 25 rules · source/config/dependency inputs · console/JSON/HTML · eligible MCP03/MCP06 fixes.” Add the workflow: file or folder → findings with evidence → reviewer action.

Say: “The project expanded beyond the initial two-threat design. The catalog describes implemented checks; our evaluation covers a narrower subset.” **Time:** 40 seconds.

5 · How the scanner works

On slide: Discovery → parsing and linked model → analyzers → filtering/deduplication → report. Branch to conservative fixes and verification. Label Python AST and limited JS/TS structural parsing separately.

Say: “The analyzers work on extracted structure and bounded data-flow evidence. Extraction coverage is a prerequisite for detection.” **Time:** 50 seconds.

6 · Improving detection without flooding the report

On slide: Two concrete changes: programmatic Python registration and derived metadata recover missed tools; executable-call analysis separates script operations from quoted examples and recognizes narrow safe argument patterns. Add HTTP text propagation as a third example if space permits.

Say: “We worked on both missed positives and false alarms, with positive and negative controls. These changes preserve stable rule IDs.” **Time:** 45 seconds.

7 · Live demonstration: from server path to HTML

On slide: One real command and a cropped report screenshot. Switch to the terminal and browser during delivery.

```
python demo.py .\tests\samples\vulnerable_tool_poisoning.py --html scan-report.html
Start-Process .\scan-report.html
```

Show: the source path; one finding's location, evidence, severity, rule, and recommendation; the number of recognized tools. State that the fixture is intentionally vulnerable. The command performs a fresh scan; the HTML and terminal use the same result.

Say: “The same command accepts an existing server directory. It reads source statically; it does not start the MCP server.” **Time:** 90 seconds.

8 · Reviewable remediation

On slide: Before → dry-run diff → backup/apply → verification. State: “Eligible Python MCP03/MCP06 findings only; ambiguous cases remain for review.”

Say: “A partial fix is reported as partial. Other risk categories are report-only, and verification does not certify the entire server as safe.” Use a verified example or dry-run output; do not promise that all findings disappear. **Time:** 35 seconds.

9 · How we evaluated the scanner

On slide: A three-stage timeline: original reserved run → exposed development rounds 008–010 → frozen new reserve. Under it: "Explicit artifact/rule labels; fixed source revisions; saved predictions and hashes; deterministic runs." Define recall in one line: "detected positives / all labeled positives."

Say: "Once we use a test set to guide improvements, it becomes development data. We report the subsequent reserved run separately." **Time:** 50 seconds.

10 · Original reserved result

On slide: "33 artifacts · seven repositories · 50 labels · eight categories / ten rules." Two metrics: **92.9% precision**, **43.3% recall**. Small confusion matrix: TP13 / FP1 / FN17 / TN19. Footnote: "Labeled scope only; 37 additional predictions unscored."

Say: "Precision was promising within reviewed labels, but 17 of 30 positive labels were missed. That made recall our improvement priority." Do not call this the current scanner's unseen performance after tuning. **Time:** 45 seconds.

11 · Development results: fewer missed signals

On slide: Insert final_results_development.svg or its PNG. Headline: **"Three more positives detected; one false alarm removed."** Show **43.3% → 53.3% recall (+10 percentage points)** and **17 → 14 misses** on the unchanged 50-label set. Tag prominently: **Development retest**.

Say: "These are paired improvements on the same labels. The smaller adapted and synthetic development sets also reached full recall, but those scores are not independent validation." Put the round 009/010 details in an appendix. **Time:** 50 seconds.

12 · New reserve: extraction coverage limited recall

On slide: "10 native files · one shared JSON-RPC implementation family · one MCP03 rule." Show **0/2 positives detected**, **0/8 false alarms**, **0 tools recognized**, **0 scan errors**. Precision: **undefined**. Diagram: handwritten `tools/list` metadata → extraction gap → metadata analyzer not reached.

Say: "The held-out test exposed an unsupported registration style. Both positives stay counted as misses. This set is too small and narrow to estimate overall recall; zero false alarms here does not prove reliable negative discrimination." Keep this in the main deck. **Time:** 65 seconds.

13 · Engineering verification and traceability

On slide: "172 tests passed at scanner freeze; 180 after demo/report additions." Include "unchanged labeled outcomes in rounds 001–007" and "saved manifests, hashes, predictions, code snapshot, consumed marker."

Say: "These checks support reproducibility and software correctness within tested behavior. They are separate from detection accuracy. No scanner change followed the new reserved run." **Time:** 35 seconds.

14 · Limitations and next steps

On slide: Three priorities: handwritten JSON-RPC extraction; broader independently reviewed labels and repository/registration coverage; new frozen reserve after repairs. Small footer: "Static heuristics; curated small sets; limited runtime and whole-project validation."

Say: "820 unadmitted text records remain as candidate material. They need review and deduplication; they are not automatically independent tests. Labels in the present study still need independent human adjudication." **Time:** 45 seconds.

15 · Delivered contribution

On slide: "A working scanner. Reviewable security evidence. Reproducible evaluation." Add: "Measured development gains; next priority: tool extraction coverage." Include a link to the results handout and Q&A.

Say: "We can demonstrate the full workflow today, explain the measured improvements, and identify exactly where the next recall gains need to come from." **Time:** 20 seconds.

The timing totals approximately **10 minutes 55 seconds**, including the live demo, leaving about one minute for transitions in a 12-minute slot before questions.

Appendix material and preparation

Appendix slides

Appendix	Contents	Source
A · Rule coverage	Full 16-category/25-rule catalog; indicate the eight categories/ten rules in the original reserved evaluation.	`mcp_scanner/catalog.py`; original validation report
B · Development details	Round 008: TP16/FP0/FN14/TN20; round 009: 6/0/0/6; round 010: 6/0/0/6. Include their distinct data origins and before scores.	`Followup_Validation_Report.md`
C · Architecture detail	Real module/component diagram using old slides 9, 11, and 13; updated technology table from old slide 14.	`README.md`; scanner/parser/analyzer/reporter modules
D · Remediation limits	Dry-run, backup, syntax validity, idempotence, eligible categories, and partial fixes.	`evaluation/followup/cli-checks.json`
E · Evaluation protocol	Label definition, exclusions, no exact-byte overlap against 290 admitted hashes, correlated-template limitation, and pending independent label review.	Both validation reports and frozen manifests

Strong phrasing that the evidence supports

Use this wording	Avoid this wording
"Implemented checks across 16 catalog categories."	"Validated protection against all 16 attack types."
"Recall increased by 10 percentage points on the same development labels."	"The scanner is 10% more accurate on unseen servers."
"100% recall on these small development sets."	"100% detection accuracy."
"92.9% precision within the original reviewed labels."	"92.9% of all findings are confirmed vulnerabilities."
"The new reserve exposed a handwritten JSON-RPC extraction gap."	"The failed cases do not count because they are unsupported."
"Static inspection of a server's source code."	"We ran penetration tests against a live server."
"Conservative automatic remediation for eligible findings."	"The scanner automatically secures the server."
"Tested workflow with reproducible evaluation artifacts."	"Production-ready protection" or "better than other scanners."

Live-demo rehearsal

1. Open PowerShell in the actual repository root and inspect the included sample before the presentation. Do not paste a fictional `C:\path\to\mcp-project` target.
2. Run the command on slide 7. Findings return exit code 1 by design; this is not a demo failure. Check the actual file/tool counts before describing the result.
3. Open the generated report with its terminal link or `Start-Process`. Explain one finding well instead of scrolling through every finding.
4. If showing remediation, rehearse it on a disposable copy and keep any remaining findings visible. A dry-run is sufficient for the main presentation.
5. Keep a recent HTML export and screenshot available as a clearly labeled fallback. The older `deliverables/Demo.html` is an illustrative walkthrough and should not be presented as a fresh target scan.
6. Recheck any results that change before submission. Do not rerun a consumed reserve and describe the rerun as fresh independent validation.

Likely professor questions

Why is recall still limited? Extraction and bounded static reasoning limit which metadata and flows reach the rules. The new reserve gives a specific example: handwritten JSON-RPC registration. Other old-set misses remain; the gap is not solved yet.

Why did the new reserve score worse than development? It uses a different registration style and a narrow, correlated family. Development improvements did not cover that style. Report the two measurements separately rather than averaging them.

Are the labels ground truth? They are explicit reviewed labels with source rationales, but independent human adjudication remains pending. Some historical rule mappings may need review; they were not changed to fit predictions.

What is original about the project? The implemented integration of MCP-aware extraction, rule-based evidence, bounded flow analysis, conservative remediation, and a reproducible evaluation workflow. Do not claim a new general static-analysis algorithm or superiority without a controlled comparison.

Does zero findings mean safe? No. Check recognized tools, scan errors, supported source patterns, and rule scope. Static heuristics cannot prove the absence of vulnerabilities.

Files to use when editing

- Professor-facing results and its editable Markdown source.
- Development chart, SVG and PNG, ready to insert into PowerPoint.
- Original validation report and follow-up validation report.
- Individual before/after decisions and new reserved metrics.

For current evaluation facts, use these records instead of the older `PRESENTATION_HANDOFF_FOR_GEMINI.md` and `PRESENTER_NOTES.md`: their descriptions of an unused holdout or the demo workflow are stale. Historical reports should retain their original dates and results.
